BACKEND DEVELOPER – TECHNICAL ASSESSMENT

Project & Task Management API

Objective:

Develop a RESTful API that allows users to register, log in, manage projects and tasks, filter and paginate tasks, and view dashboard statistics.

1. Authentication (MySQL)

Implement the following APIs:

- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/auth/me`
- `PUT /api/auth/profile`

Requirements:

- Store user information in MySQL.
- Hash passwords using bcrypt.
- Implement JWT-based authentication.
- Validate user inputs and handle duplicate email addresses.
- Restrict profile access and updates to the authenticated user.

2. Project Management (MySQL)

Implement CRUD operations:

- `POST /api/projects`
- `GET /api/projects`
- `GET /api/projects/:id`
- `PUT /api/projects/:id`
- `DELETE /api/projects/:id`

Requirements:

- Store project information in MySQL.
- Support project statuses: `active`, `completed`, and `archived`.
- Ensure users can access and manage only their own projects.
- Implement appropriate validation and error handling.

3. Task Management (MongoDB)

Implement CRUD operations:

- `POST /api/tasks`
- `GET /api/tasks`
- `GET /api/tasks/:id`
- `PUT /api/tasks/:id`
- `DELETE /api/tasks/:id`

Requirements:

- Store task information in MongoDB using Mongoose.
- Associate each task with a project and its owner.
- Support task statuses: `todo`, `in_progress`, and `completed`.
- Support priorities: `low`, `medium`, and `high`.
- Validate task data and project ownership.
- Handle invalid IDs and non-existent tasks appropriately.

4. Filtering and Pagination

Enhance `GET /api/tasks` to support:

- Pagination using `page` and `limit`.
- Filtering by `status`, `priority`, and `projectId`.

Example:

```
GET /api/tasks?page=1&limit=10&status=todo&priority=high
```

Return task data along with pagination details, including total records and total pages.

5. Dashboard API

Implement:

```
GET /api/dashboard
```

Return the following statistics for the authenticated user:

- Total projects
- Active projects
- Completed projects
- Total tasks
- Pending tasks
- Completed tasks
- High-priority tasks

Retrieve the information from the appropriate databases.

6. Validation and Error Handling

- Validate all incoming request data.
- Use consistent success and error response formats.
- Implement centralized error-handling middleware.
- Handle authentication errors, duplicate emails, database errors, invalid IDs, and missing resources.
- Use appropriate HTTP status codes.

7. Project Structure and Configuration

- Organize code into routes, controllers, models, middleware, services, and configuration files.
- Use environment variables for database credentials, JWT secrets, and application settings.
- Include a `.env.example` file.
- Do not hardcode sensitive credentials.

8. Documentation and Testing

Include a `README.md` covering:

- Project overview and technologies used.
- Installation and setup instructions.
- MySQL and MongoDB configuration.
- Environment variable setup.
- Available APIs and sample requests/responses.

Test authentication APIs, project and task CRUD operations, pagination, filtering, validation, and unauthorized access using Postman, Thunder Client, or an equivalent testing tool.

9. Submission Requirements

Submit the completed project through a Git repository containing:

- Source code
- `README.md`
- `.env.example`
- Database setup instructions
- API documentation

Do not upload `node_modules`, `.env` files, passwords, JWT secrets, or other sensitive credentials.

Important Instructions:

Focus on clean, readable, secure, and maintainable code. You should be prepared to explain your implementation, database design decisions, authentication flow, and API functionality during the technical discussion.

Good luck!